One-Dimensional Linear Advection of a Wave

First-order upwind method and explicit Euler time integration

Step-by-step explanation with a Julia implementation

Domain	Velocity	Initial amplitude	Final time
[0, 1]	a = 1	1	t = 0.5

This document explains the governing equation, initial and boundary conditions, numerical discretization, CFL stability criterion, algorithm, Julia code, and interpretation of the computed solution.

1. Problem objective

The objective is to simulate a smooth wave transported from left to right through a one-dimensional periodic domain. For the exact solution, the profile preserves its shape and amplitude and is translated at a constant velocity. The numerical solution uses a first-order upwind approximation in space and the explicit Euler method in time.

- Spatial domain: 0 <= x < 1.
- Constant advection velocity: a = 1, so information travels from left to right.
- Initial condition: a Gaussian pulse centered at x = 0.25 with maximum amplitude equal to 1.
- Boundary condition: periodic, so the value leaving through x = 1 re-enters through x = 0.
- Final time: t = 0.5, corresponding to an exact displacement of 0.5.

2. Governing equation

The one-dimensional linear advection equation with constant velocity is

$$\partial u/\partial t + a\, \partial u/\partial x = 0.$$

The scalar u(x,t) can represent concentration, temperature, a passive marker, or another transported quantity. There are no source, diffusion, or reaction terms in this model.

For constant a, the characteristics satisfy x - at = constant. Therefore, the exact solution is simply the translated initial condition:

$$u(x,t) = u_0(x - at).$$

With a = 1 and t = 0.5, every point of the profile moves 0.5 units to the right.

3. Initial condition

The initial wave is a Gaussian pulse:

$$u(x,0) = \exp[-((x - x_c)/\sigma)^2], \quad x_c = 0.25, \quad \sigma = 0.05.$$

At x = x_c, the exponent is zero and exp(0) = 1. Thus, the initial maximum amplitude is exactly 1. The parameter σ controls the pulse width: a smaller value produces a narrower wave.

4. Spatial grid

The interval [0,1] is divided into N uniformly spaced points. Because the domain is periodic, x = 1 is not stored as a separate point; it is equivalent to x = 0.

$$\Delta x = (x_max - x_min)/N.$$

For N = 200, Δx = 0.005, and the stored coordinates are 0, 0.005, ..., 0.995.

5. First-order upwind discretization

Because a > 0, information arriving at node i comes from the node immediately to its left, i - 1. The spatial derivative is therefore approximated by a backward difference:

$$(\partial u/\partial x)_i^n \approx (u_i^n - u_{i-1}^n)/\Delta x.$$

This is the upwind principle: the stencil is selected from the upstream side. If a were negative, information would arrive from the right and a forward difference would be required.

6. Explicit Euler time discretization

The time derivative is approximated with a forward difference:

$$(\partial u/\partial t)_i^n \approx (u_i^{n+1} - u_i^n)/\Delta t.$$

Substituting the spatial and temporal approximations into the differential equation gives

$$(u_i^{n+1} - u_i^n)/\Delta t + a(u_i^n - u_{i-1}^n)/\Delta x = 0.$$

Solving for the unknown value at the new time level results in the update equation used in the code:

$$u_i^{n+1} = u_i^n - C(u_i^n - u_{i-1}^n), \quad C = a\Delta t/\Delta x.$$

7. CFL stability condition

For the explicit upwind method, stability requires the Courant number C to satisfy

$$0 <= C = |a|\Delta t/\Delta x <= 1.$$

The code adopts CFL = 0.8 and defines Δt = CFL Δx/|a|. With a = 1 and Δx = 0.005, the time step is Δt = 0.004. A value below one ensures that the numerical domain of dependence contains the physical domain of dependence.

8. Periodic boundary condition

At the first grid point, the upstream neighbor would have index zero, which does not exist in Julia. Periodicity means that the left neighbor of the first point is the last point of the array. Therefore,

$$u_1^{n+1} = u_1^n - C(u_1^n - u_N^n).$$

This treatment allows the profile to circulate through the domain without prescribing an inflow waveform at x = 0.

9. Algorithm

1. Create the uniform spatial grid in [0,1).
2. Initialize the Gaussian pulse with amplitude 1.
3. Compute Δt from the chosen CFL number.
4. Copy the current solution into u_old at the beginning of each time step.
5. Update the internal points with the first-order upwind formula.
6. Update the first point using the periodic boundary condition.
7. Advance the simulation time and repeat until t = 0.5.
8. Plot the initial and final numerical profiles.

10. Complete Julia code

```
using Plots
```

```
# ============================================================
# One-dimensional linear advection equation:
#
#     ∂u/∂t + a ∂u/∂x = 0
#
# Numerical method:
# - First-order upwind scheme in space
# - Explicit Euler method in time
# - Periodic boundary conditions
# ============================================================

# Spatial domain
x_min = 0.0
x_max = 1.0

# Constant advection velocity
# A positive value transports the wave from left to right.
a = 1.0

# Number of grid points
number_of_points = 200

# Spatial step
# The point x = 1 is not stored because the domain is periodic.
dx = (x_max - x_min) / number_of_points

# Grid coordinates: 0, dx, 2dx, ..., 1 - dx
x = collect(range(x_min, x_max - dx, length = number_of_points))

# Courant number
CFL = 0.8

# Time step obtained from the CFL condition
dt = CFL * dx / abs(a)

# Final simulation time
final_time = 0.5

# Initial Gaussian wave with maximum amplitude equal to 1
wave_center = 0.25
wave_width = 0.05
u = exp.(-((x .- wave_center) ./ wave_width).^2)

# Store the initial profile for comparison
u_initial = copy(u)

# Initial time
time = 0.0

# Explicit time integration loop
while time < final_time

    # Adjust the last time step to reach final_time exactly
    current_dt = min(dt, final_time - time)

    # Courant number for the current time step
    C = a * current_dt / dx

    # Keep the old solution unchanged during the update
    u_old = copy(u)

    # Upwind update for the internal grid points
    for i in 2:number_of_points
        u[i] = u_old[i] - C * (u_old[i] - u_old[i - 1])
    end

    # Periodic boundary condition at the first grid point
    u[1] = u_old[1] - C * (u_old[1] - u_old[end])

    # Advance the physical time
    time += current_dt
end

# Plot the initial and final solutions
plot(
    x,
    u_initial,
    label = "Initial wave",
    linewidth = 2,
    linestyle = :dash,
    xlabel = "x",
    ylabel = "u(x,t)",
    title = "Linear Advection with Explicit Upwind Scheme",
    xlims = (0.0, 1.0),
    ylims = (0.0, 1.1),
    legend = :topright
```

```
)

plot!(
    x,
    u,
    label = "Numerical solution at t = $(final_time)",
    linewidth = 3
)

display(current())
```

11. Explanation of the main code sections

Grid construction: The grid contains 200 points from 0 to 0.995. Excluding x = 1 avoids duplicating the periodic point x = 0.

Initial wave: The vectorized expression exp.(-((x .- wave_center) ./ wave_width).^2) evaluates the Gaussian at every grid point. The dot operators perform elementwise operations.

Old and new solutions: The copy u_old is essential for an explicit method. Every value at time level n+1 must be computed only from values at time level n.

Internal update: For a positive velocity, each point uses its own old value and the old value immediately to its left.

Periodic update: The first point uses u_old[end] as its upstream neighbor.

Final-time adjustment: The min operation prevents the last step from advancing beyond the requested final time.

12. Numerical result

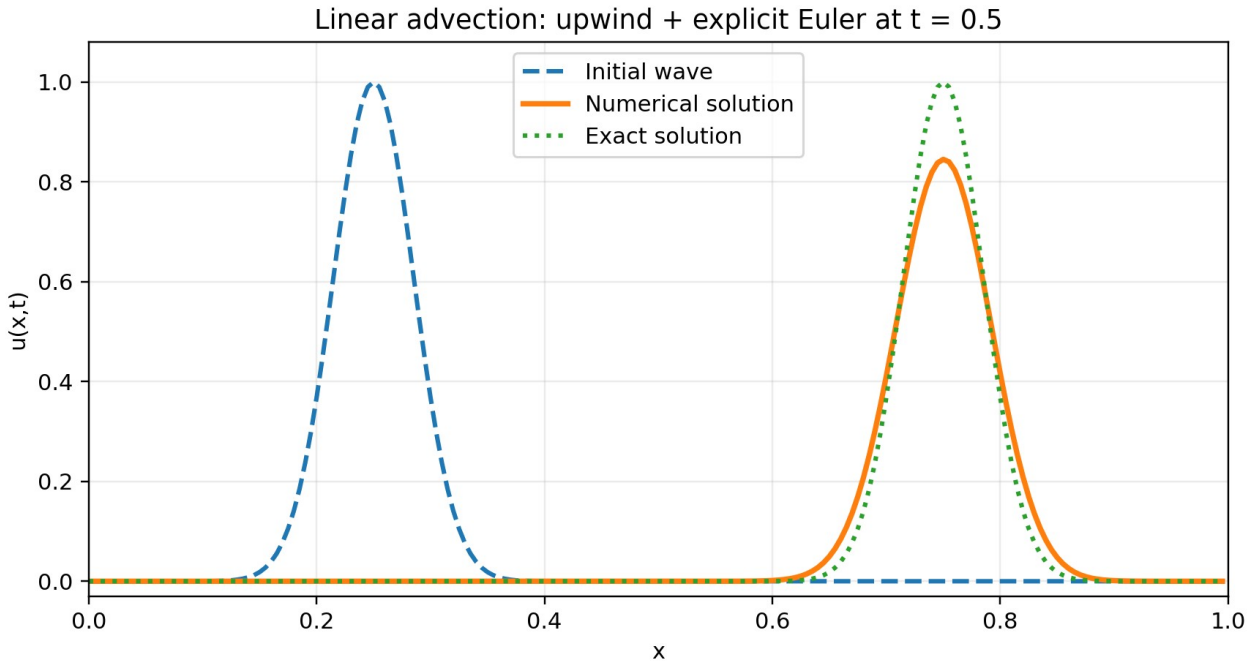


Figure 1. Initial Gaussian pulse, first-order upwind solution, and exact translated profile at t = 0.5.

The exact wave center moves from x = 0.25 to x = 0.75. The numerical solution reaches approximately the correct location, but its peak is lower and its width is larger. This deformation is caused by numerical diffusion introduced by the first-order upwind discretization.

13. Accuracy and numerical diffusion

The method is first-order accurate in both space and time. It is robust and simple, but smooth gradients become progressively smeared. This effect is not physical diffusion; it is a truncation error of the numerical scheme.

The modified-equation interpretation shows that, for a > 0, the leading artificial-diffusion coefficient is approximately

$$\nu_{num} \approx a\Delta x(1 - C)/2.$$

Consequently, reducing Δx generally decreases diffusion. For a fixed grid, increasing C toward 1 also reduces the leading diffusion term. In the special case C = 1, the discrete solution is shifted exactly by one grid point per time step for positive a, although this observation does not remove the general first-order limitations of the method.

14. Suggested numerical experiments

- Change number_of_points from 200 to 50, 100, 400, and 800 and compare the peak attenuation.
- Compare CFL values 0.2, 0.5, 0.8, and 1.0 while preserving the stability limit.
- Set final_time = 1.0. With periodic boundaries and a = 1, the exact wave returns to its initial position.
- Change a to -1 and modify the upwind stencil so that each point uses the value to its right.
- Replace the Gaussian pulse with a square wave to observe stronger smearing near discontinuities.

15. Key conclusions

- The linear advection equation transports the initial profile at velocity a.
- For a > 0, the first-order upwind stencil uses the point to the left.
- Explicit Euler requires all updates to use the previous time level.
- The explicit upwind method is stable when the Courant number does not exceed one.
- Periodic boundaries connect the last grid point to the first.
- The computed wave travels at approximately the correct speed but loses amplitude because of numerical diffusion.